

Университет ИТМО

Факультет/направление: ПИиКТ/СППО

Лабораторная работа №2

Блочный кэш страниц (VT Page Cache)

Вариант: **Optimal** (с подсказкой)

Студент: **Брагин Роман**, группа **Р3316**

Преподаватель: **Сорокин Артём Николаевич**

Санкт-Петербург, 2025

1 Текст задания (кратко)

Необходимо реализовать блочный кэш страниц в user-space в виде динамической библиотеки на C. Библиотека должна предоставлять API, аналогичный системным вызовам: открытие/закрытие файла, чтение, запись, позиционирование, синхронизация. Операции чтения/записи на диск должны выполняться в обход page cache ОС.

Вариант: **Optimal**. Для него требуется предоставить пользователю возможность подсказать момент следующего обращения к данным (например, через отдельную функцию `advice`).

2 Обзор реализации

2.1 Структура проекта

- `lib/vtpc.c`, `lib/vtpc.h` — реализация библиотеки кэша страниц.
- `test/` — тесты корректности (поведение сравнивается с `libc`).
- `bench/io-load.c` — IO-нагрузчик для сравнения производительности `libc` и `vtpc`.

2.2 Реализованный API

Библиотека предоставляет функции:

- `vtpc_open`, `vtpc_close`, `vtpc_read`, `vtpc_write`, `vtpc_lseek`, `vtpc_fsync`
- `vtpc_advice(fd, offset, next_use)` — подсказка для **Optimal**
- алиасы `lab2_*` для соответствия формату задания (например, `lab2_open`, `lab2_advice`)

2.3 Кэш страниц

- Кэш состоит из фиксированного числа страниц (`VTPC_CACHE_PAGES`), каждая страница имеет размер (`VTPC_PAGE_SIZE`).
- Данные страниц хранятся в выровненных буферах (через `posix_memalign`) для корректной работы с `O_DIRECT`.
- При `read/write` обрабатываются произвольные размеры и смещения: операция раскладывается на затронутые страницы, а работа с диском выполняется страничными блоками.
- Модифицированные страницы помечаются как `dirty` и сбрасываются на диск при вытеснении/ `fsync`/ `close`.

2.4 Обход page cache ОС

При открытии файла библиотека пытается использовать `O_DIRECT`. Если файловая система не поддерживает `O_DIRECT`, библиотека автоматически открывает файл без него, при этом старается минимизировать влияние page cache ОС (best-effort через `posix_fadvise(..., DONTNEED)`).

2.5 Политика вытеснения Optimal

- Для каждой страницы поддерживается значение `next_use` — “время” следующего обращения.
- При необходимости вытеснения выбирается страница с максимальным `next_use` (Belady/Optimal).
- При равенстве `next_use` используется тай-брейк по давности доступа (LRU).
- Пользователь задаёт `next_use` через `vtpc_advice/lab2_advice`.

3 Проверка корректности

Корректность проверялась тестами из каталога `test/`:

- `test_basic`
- `test_seq`
- `test_random`

Тесты запускаются через CTest:

```
make test
#
ctest --test-dir build/test --output-on-failure
```

4 Замеры производительности (до/после)

4.1 Методика

Для замеров использовался нагрузчик `bench/io-load` со следующими параметрами:

- блок: 4096 байт
- количество операций: 20000
- диапазон: 0–262144 (256 КиБ, горячий набор данных)
- порядок: `random`
- сравнение:
 - `libc`: `pread/pwrite` с `O_DIRECT=on`
 - `vtpc`: библиотека кэша, `VTPC_PAGE_SIZE=4096`, `VTPC_CACHE_PAGES=64`

4.2 Результаты

Комментарий. На “горячем” наборе данных (256 КиБ) кэш позволяет практически полностью убрать повторные обращения к диску: страницы читаются/записываются в RAM, а на диск сбрасываются агрегированно.

Сценарий	libc O_DIRECT (MB/s)	vtpc (MB/s)	Ускорение
Random read (256 KiB hot set)	21.60	5032.23	$\approx 233\times$
Random write (256 KiB hot set)	17.51	4464.67	$\approx 255\times$

Таблица 1: Пример сравнения производительности до/после внедрения page cache

5 Вывод

В рамках работы реализован пользовательский page cache в виде динамической библиотеки на C с обходом page cache ОС и политикой вытеснения Optimal (Belady) с подсказками следующего обращения. Тесты корректности проходят, а на сценариях с повторным доступом к одному и тому же диапазону данных наблюдается существенный выигрыш по производительности.